

University of Chittagong
Department of Computer Science & Engineering

Machine Learning

CSE-816

Submitted by:

Jannatul Ferdous

Student ID: 21701059

Submitted to:

Dr. Mohammad Osiur Rahman

Department of Computer Science & Engineering

University of Chittagong

14th September 2026

Contents

1 KNN (K-Nearest Neighbors) 2

2 SVM (Support Vector Machine) 8

3 K-Means Clustering 15

4 Decision Tree 21

5 Perceptron — Apple vs Orange (Hagan, Ch. 4) 28

Chapter 1

KNN (K-Nearest Neighbors)

Introduction

KNN is a supervised classification algorithm based on a simple idea: to classify a new point, look at its K closest neighbors in the training data and let them vote. KNN does not really "train" in the usual sense — it simply stores the training data and does all its work at prediction time.

Reading the Dataset

The built-in Height-Weight dataset (a synthetic but realistic dataset generated in code) is used, with the goal of predicting Gender from Height and Weight:

```
def generate_height_weight_dataset(n=200, seed=42):
    rng = np.random.default_rng(seed)
    male_height = rng.normal(175, 7, n//2)
    male_weight = rng.normal(78, 10, n//2)
    female_height = rng.normal(162, 6, n - n//2)
    female_weight = rng.normal(60, 8, n - n//2)
    ...
    return DataFrame with columns Height(cm), Weight(kg), Gender
```

Training

Features are scaled (StandardScaler) because KNN relies on distance, then the KNeighborsClassifier is fit on the training split. K, the distance metric, and the test split are all adjustable from the UI sidebar:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KNeighborsClassifier(n_neighbors=k_value, metric=distance_metric)
model.fit(X_train, y_train)
```

How the UI Gives Output

- Dataset preview table + Accuracy, Confusion Matrix, Classification Report after training
- Type a new Height and Weight into number-input boxes → click Predict → get Gender + probability bar chart

How to Run This Application

1) Keep the `knn_app/` folder on your laptop
2) Open a terminal and go into that folder (`cd knn_app/`)
3) Install the libraries: `pip install -r requirements.txt`
4) Run the app: `streamlit run app.py`
5) A browser tab opens automatically (if not, go to http://localhost:8501)

Implementation

```
"""
KNN (K-Nearest Neighbors) Classifier - Interactive UI
-----------------------------------------------------
This application is a user interface (UI) built with Streamlit.
No CLI is used here  all inputs are provided through buttons, input boxes, and
    forms.

How to run (type in terminal):
   streamlit run app.py

Then open http://localhost:8501 in your browser to see the interface.
"""

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
   accuracy_score,
   confusion_matrix,
   classification_report,
)
```

```
# ------------------------------------------------------------
# A simple, easy-to-understand dataset: Gender Classification using Height-
    Weight
# (Generated to follow a realistic-like distribution)
# ------------------------------------------------------------
@st.cache_data
def generate_height_weight_dataset(n=200, seed=42):
    rng = np.random.default_rng(seed)
    n_male = n // 2
    n_female = n - n_male

    male_height = rng.normal(175, 7, n_male) # average height 175 cm
    male_weight = rng.normal(78, 10, n_male) # average weight 78 kg

    female_height = rng.normal(162, 6, n_female) # average height 162 cm
    female_weight = rng.normal(60, 8, n_female) # average weight 60 kg

    data = pd.DataFrame(
        {
            "Height(cm)": np.round(np.concatenate([male_height, female_height])
                , 1),
            "Weight(kg)": np.round(np.concatenate([male_weight, female_weight])
                , 1),
            "Gender": ["Male"] * n_male + ["Female"] * n_female,
        }
    )
    return data.sample(frac=1, random_state=seed).reset_index(drop=True)


# ------------------------------------------------------------
# Page setup
# ------------------------------------------------------------
st.set_page_config(page_title="KNN Classifier", layout="wide")
st.title(" K-Nearest Neighbors (KNN) Classifier")
st.caption("Machine Learning Lab Experiment  Interactive UI (No CLI)")

# ------------------------------------------------------------
# Step 1: Reading the dataset (selectable from the sidebar)
# ------------------------------------------------------------
st.sidebar.header(" Settings")

data_source = st.sidebar.radio(
    "Step 1: Choose a dataset",
    ["Height-Weight Dataset (Gender Classification)", "Upload your own CSV"],
)

if data_source == "Height-Weight Dataset (Gender Classification)":
    df = generate_height_weight_dataset()
    feature_cols = ["Height(cm)", "Weight(kg)"]
    target_col = "Gender"
```

```
    target_names = sorted(df[target_col].unique().tolist())
else:
    uploaded_file = st.sidebar.file_uploader("Upload a CSV file", type=["csv
        "])
    if uploaded_file is None:
        st.info(" Please upload a CSV file from the left, or choose the Height
            -Weight Dataset.")
        st.stop()
    df = pd.read_csv(uploaded_file)
    all_cols = list(df.columns)
    target_col = st.sidebar.selectbox("Which column is the Target/Label (the
        one you want to predict)?", all_cols)
    feature_cols = st.sidebar.multiselect(
        "Which columns are Features (must be numeric)?",
        [c for c in all_cols if c != target_col],
        default=[c for c in all_cols if c != target_col],
    )
    target_names = sorted(df[target_col].astype(str).unique().tolist())

# ------------------------------------------------------------
# Step 2: KNN hyperparameters
# ------------------------------------------------------------
st.sidebar.markdown("---")
st.sidebar.subheader("Step 2: KNN Parameters")
k_value = st.sidebar.slider("K (Number of Neighbors)", min_value=1, max_value
    =20, value=5)
test_size = st.sidebar.slider("Test Data Fraction (%)", min_value=10,
    max_value=50, value=20, step=5) / 100
distance_metric = st.sidebar.selectbox("Distance Metric", ["euclidean", "
    manhattan", "minkowski"])

# ------------------------------------------------------------
# Dataset preview
# ------------------------------------------------------------
st.subheader(" 1. Dataset Preview")
c1, c2 = st.columns([3, 1])
with c1:
    st.dataframe(df.head(10), use_container_width=True)
with c2:
    st.metric("Total Rows", df.shape[0])
    st.metric("Total Features", len(feature_cols))
    st.metric("Number of Classes", len(target_names))

# ------------------------------------------------------------
# Step 3: Train button
# ------------------------------------------------------------
st.subheader(" 2. Train the Model")
train_clicked = st.button("Train KNN Model", type="primary")

if "trained" not in st.session_state:
    st.session_state.trained = False
```

```
if train_clicked:
    X = df[feature_cols].values
    y = df[target_col].values

    # Feature scaling (KNN is distance-based, so scaling is essential)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=test_size, random_state=42, stratify=y
    )

    model = KNeighborsClassifier(n_neighbors=k_value, metric=distance_metric)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    # Saved in session_state so it can be reused across the UI
    st.session_state.trained = True
    st.session_state.model = model
    st.session_state.scaler = scaler
    st.session_state.acc = acc
    st.session_state.y_test = y_test
    st.session_state.y_pred = y_pred
    st.session_state.feature_cols = feature_cols
    st.session_state.target_names = target_names

# ------------------------------------------------------------
# Showing results
# ------------------------------------------------------------
if st.session_state.trained:
    st.success(f" Model training complete! Accuracy = {st.session_state.acc
        *100:.2f}%")

    col1, col2 = st.columns(2)

    with col1:
        st.markdown("**Confusion Matrix**")
        cm = confusion_matrix(st.session_state.y_test, st.session_state.y_pred
            )
        fig, ax = plt.subplots()
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
        ax.set_xlabel("Predicted")
        ax.set_ylabel("Actual")
        st.pyplot(fig)

    with col2:
        st.markdown("**Classification Report**")
        report = classification_report(
```

```
            st.session_state.y_test, st.session_state.y_pred, output_dict=True
        )
        st.dataframe(pd.DataFrame(report).transpose(), use_container_width=
            True)

    # ------------------------------------------------------
    # Step 4: Prediction with new data (Live Prediction UI)
    # ------------------------------------------------------
    st.subheader(" 3. Predict with New Input")
    st.write("Type values directly into the boxes below  you can enter your
        own input here.")

    input_values = []
    cols = st.columns(len(st.session_state.feature_cols))
    for i, feat in enumerate(st.session_state.feature_cols):
        mean_v = float(df[feat].mean())
        with cols[i]:
            val = st.number_input(
                feat,
                value=round(mean_v, 1),
                step=0.1,
                format="%.1f",
            )
        input_values.append(val)

    if st.button("Predict with this Input"):
        X_new = st.session_state.scaler.transform([input_values])
        pred = st.session_state.model.predict(X_new)[0]
        proba = st.session_state.model.predict_proba(X_new)[0]

        st.markdown(f"###  Prediction Result: **{pred}**")
        proba_df = pd.DataFrame(
            {"Class": st.session_state.model.classes_, "Probability": proba}
        ).sort_values("Probability", ascending=False)
        st.bar_chart(proba_df.set_index("Class"))
else:
    st.info(" Click the 'Train KNN Model' button above first.")
```

Chapter 2

SVM (Support Vector Machine)

Introduction

SVM is a supervised classifier that finds the boundary between classes with the maximum possible margin. The points closest to the boundary — the ones that actually decide where it sits — are called Support Vectors.

Reading the Dataset

The same Height-Weight dataset as KNN is used, for consistency.

Training

Kernel (linear/rbf/poly/sigmoid), C (regularization strength), and Gamma are chosen from the UI. The kernel decides the shape of the boundary — rbf/poly can curve, linear cannot:

```
model = SVC(kernel=kernel, C=c_value, gamma=gamma_option, probability=True)
model.fit(X_train, y_train)
```

How the UI Gives Output

- Accuracy, Confusion Matrix, Classification Report
- A Decision Boundary plot showing exactly how SVM separates the two classes in 2D
- Number-input boxes for Height/Weight → Predict → Gender + probability chart

How to Run This Application

1) Keep the `svm_app/` folder on your laptop
2) Open a terminal and go into that folder (`cd svm_app/`)
3) Install the libraries: `pip install -r requirements.txt`
4) Run the app: `streamlit run app.py`
5) A browser tab opens automatically (if not, go to http://localhost:8501)

Implementation

```
"""
SVM (Support Vector Machine) Classifier - Interactive UI
------------------------------------------------------------
This is a graphical user interface built with Streamlit.
No CLI is used here  everything is done with buttons, dropdowns,
and typeable input boxes.

How to run (in your terminal):
    streamlit run SVM.py

Then open http://localhost:8501 in your browser.
"""

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
)

# ------------------------------------------------------------
# A simple, easy-to-understand built-in dataset:
# Predict Gender from Height and Weight
# ------------------------------------------------------------
@st.cache_data
def generate_height_weight_dataset(n=200, seed=42):
    rng = np.random.default_rng(seed)
    n_male = n // 2
    n_female = n - n_male

    male_height = rng.normal(175, 7, n_male) # average height 175 cm
    male_weight = rng.normal(78, 10, n_male) # average weight 78 kg

    female_height = rng.normal(162, 6, n_female) # average height 162 cm
    female_weight = rng.normal(60, 8, n_female) # average weight 60 kg

    data = pd.DataFrame(
        {
            "Height(cm)": np.round(np.concatenate([male_height, female_height])
                , 1),
```

```
            "Weight(kg)": np.round(np.concatenate([male_weight, female_weight])
                , 1),
            "Gender": ["Male"] * n_male + ["Female"] * n_female,
        }
    )
    return data.sample(frac=1, random_state=seed).reset_index(drop=True)


# ------------------------------------------------------------
# Page setup
# ------------------------------------------------------------
st.set_page_config(page_title="SVM Classifier", layout="wide")
st.title(" Support Vector Machine (SVM) Classifier")
st.caption("Machine Learning Lab Experiment  Interactive UI (No CLI)")

# ------------------------------------------------------------
# Step 1: Read the dataset (choose it from the sidebar)
# ------------------------------------------------------------
st.sidebar.header(" Settings")

data_source = st.sidebar.radio(
    "Step 1: Choose a dataset",
    ["Height-Weight Dataset (Gender Classification)", "Upload your own CSV"],
)

if data_source == "Height-Weight Dataset (Gender Classification)":
    df = generate_height_weight_dataset()
    feature_cols = ["Height(cm)", "Weight(kg)"]
    target_col = "Gender"
    target_names = sorted(df[target_col].unique().tolist())
else:
    uploaded_file = st.sidebar.file_uploader("Upload a CSV file", type=["csv
        "])
    if uploaded_file is None:
        st.info(" Upload a CSV file from the left, or choose the Height-Weight
             Dataset.")
        st.stop()
    df = pd.read_csv(uploaded_file)
    all_cols = list(df.columns)
    target_col = st.sidebar.selectbox("Which column is the Target/Label (what
        you want to predict)?", all_cols)
    feature_cols = st.sidebar.multiselect(
        "Which columns are Features (must be numeric)? Pick exactly 2 for the
            decision boundary plot.",
        [c for c in all_cols if c != target_col],
        default=[c for c in all_cols if c != target_col][:2],
    )
    target_names = sorted(df[target_col].astype(str).unique().tolist())

# ------------------------------------------------------------
# Step 2: SVM hyperparameters
```

```
# ------------------------------------------------------------
st.sidebar.markdown("---")
st.sidebar.subheader("Step 2: SVM Parameters")
kernel = st.sidebar.selectbox("Kernel", ["linear", "rbf", "poly", "sigmoid"],
    index=1)
c_value = st.sidebar.slider("C (Regularization strength)", 0.01, 10.0, 1.0,
    step=0.01)
gamma_option = st.sidebar.selectbox("Gamma", ["scale", "auto"])
test_size = st.sidebar.slider("Test data split (%)", min_value=10, max_value
    =50, value=20, step=5) / 100

# ------------------------------------------------------------
# Dataset preview
# ------------------------------------------------------------
st.subheader(" 1. Dataset Preview")
c1, c2 = st.columns([3, 1])
with c1:
    st.dataframe(df.head(10), use_container_width=True)
with c2:
    st.metric("Total Rows", df.shape[0])
    st.metric("Total Features", len(feature_cols))
    st.metric("Number of Classes", len(target_names))

# ------------------------------------------------------------
# Step 3: Train button
# ------------------------------------------------------------
st.subheader(" 2. Train the Model")
train_clicked = st.button("Train SVM Model", type="primary")

if "trained" not in st.session_state:
    st.session_state.trained = False

if train_clicked:
    if len(feature_cols) < 2:
        st.error("Please select at least 2 feature columns.")
        st.stop()

    X = df[feature_cols].to_numpy(dtype=float)
    y = df[target_col].astype(str).to_numpy()

    # Feature scaling (SVM is distance/margin based, so scaling matters a lot)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=test_size, random_state=42, stratify=y
    )

    model = SVC(kernel=kernel, C=c_value, gamma=gamma_option, probability=True
        )
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    # Save everything in session_state so the UI can reuse it
    st.session_state.trained = True
    st.session_state.model = model
    st.session_state.scaler = scaler
    st.session_state.acc = acc
    st.session_state.y_test = y_test
    st.session_state.y_pred = y_pred
    st.session_state.X_train = X_train
    st.session_state.y_train = y_train
    st.session_state.feature_cols = feature_cols
    st.session_state.target_names = target_names

# ------------------------------------------------------------
# Show results
# ------------------------------------------------------------
if st.session_state.trained:
    st.success(f" Model trained successfully! Accuracy = {st.session_state.acc
        *100:.2f}%")

    col1, col2 = st.columns(2)

    with col1:
        st.markdown("**Confusion Matrix**")
        cm = confusion_matrix(st.session_state.y_test, st.session_state.y_pred
            )
        fig, ax = plt.subplots()
        sns.heatmap(cm, annot=True, fmt="d", cmap="Oranges", ax=ax)
        ax.set_xlabel("Predicted")
        ax.set_ylabel("Actual")
        st.pyplot(fig)

    with col2:
        st.markdown("**Classification Report**")
        report = classification_report(
            st.session_state.y_test, st.session_state.y_pred, output_dict=True
        )
        st.dataframe(pd.DataFrame(report).transpose(), use_container_width=
            True)

    # ------------------------------------------------------------
    # Decision boundary plot (only works cleanly with 2 features)
    # ------------------------------------------------------------
    if len(st.session_state.feature_cols) == 2:
        st.markdown("**Decision Boundary (Training Data)**")
        X_train = st.session_state.X_train
        y_train = st.session_state.y_train
        model = st.session_state.model
```

```
    x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
    y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200)
    )
    grid_labels = model.predict(np.c_[xx.ravel(), yy.ravel()])
    classes_sorted = sorted(set(grid_labels))
    label_to_int = {c: i for i, c in enumerate(classes_sorted)}
    Z = np.array([label_to_int[v] for v in grid_labels]).reshape(xx.shape)
    y_train_int = np.array([label_to_int[v] for v in y_train])

    fig2, ax2 = plt.subplots()
    ax2.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
    scatter = ax2.scatter(
        X_train[:, 0], X_train[:, 1], c=y_train_int, cmap="coolwarm",
            edgecolors="k"
    )
    ax2.set_xlabel(f"{st.session_state.feature_cols[0]} (scaled)")
    ax2.set_ylabel(f"{st.session_state.feature_cols[1]} (scaled)")
    st.pyplot(fig2)
    st.caption(
        "This plot shows how SVM separates the classes using its decision
            boundary. "
        "Axes are shown in scaled units because the model was trained on
            scaled data."
    )

# ------------------------------------------------------
# Step 4: Predict with new input (Live Prediction UI)
# ------------------------------------------------------
st.subheader(" 3. Predict with New Input")
st.write("Type values directly into the boxes below  this is where new
    input is entered.")

input_values = []
cols = st.columns(len(st.session_state.feature_cols))
for i, feat in enumerate(st.session_state.feature_cols):
    mean_v = float(df[feat].mean())
    with cols[i]:
        val = st.number_input(
            feat,
            value=round(mean_v, 1),
            step=0.1,
            format="%.1f",
        )
    input_values.append(val)

if st.button("Predict with this input"):
    X_new = st.session_state.scaler.transform([input_values])
    pred = st.session_state.model.predict(X_new)[0]
```

```
        proba = st.session_state.model.predict_proba(X_new)[0]

        st.markdown(f"###  Prediction Result: **{pred}**")
        proba_df = pd.DataFrame(
            {"Class": st.session_state.model.classes_, "Probability": proba}
        ).sort_values("Probability", ascending=False)
        st.bar_chart(proba_df.set_index("Class"))
else:
    st.info(" Click the 'Train SVM Model' button above first.")
```

Chapter 3

K-Means Clustering

Introduction

K-Means is unsupervised — it never sees any labels. It groups similar points together into K clusters purely based on their feature values.

Reading the Dataset

Only Height and Weight are given to the algorithm. The Gender column is kept aside and used only afterwards, to check how well the automatic clusters happen to match reality.

Training (an Iterative Process, Not One-Shot)

```
model = KMeans(n_clusters=k_value, random_state=42, n_init=10)
cluster_labels = model.fit_predict(X_scaled)
```

Internally: place K centroids → assign every point to its nearest centroid → move each centroid to the average of its assigned points → repeat until centroids stop moving.

How the UI Gives Output

- A cluster scatter plot with centroids marked
- An Elbow Curve (inertia vs K) to help choose a good K
- A comparison table: Cluster number vs real Gender label
- Type Height/Weight → Find Cluster → see which cluster the point falls into

How to Run This Application

1) Keep the `kmeans_app/` folder on your laptop
2) Open a terminal and go into that folder (`cd kmeans_app/`)
3) Install the libraries: `pip install -r requirements.txt`
4) Run the app: `streamlit run app.py`

5) A browser tab opens automatically (if not, go to http://localhost:8501)

Implementation

```
"""
K-Means Clustering - Interactive UI
------------------------------------------------------------
This is a graphical user interface built with Streamlit.
No CLI is used here  everything is done with buttons, dropdowns,
sliders, and typeable input boxes.

How to run (in your terminal):
    streamlit run app.py

Then open http://localhost:8501 in your browser.
"""

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# ------------------------------------------------------------
# A simple, easy-to-understand built-in dataset:
# Height and Weight (Gender is only used afterwards, to compare
# the clusters found by K-Means against the real groups  K-Means
# itself never sees the Gender column, since it is unsupervised).
# ------------------------------------------------------------
@st.cache_data
def generate_height_weight_dataset(n=200, seed=42):
    rng = np.random.default_rng(seed)
    n_male = n // 2
    n_female = n - n_male

    male_height = rng.normal(175, 7, n_male) # average height 175 cm
    male_weight = rng.normal(78, 10, n_male) # average weight 78 kg

    female_height = rng.normal(162, 6, n_female) # average height 162 cm
    female_weight = rng.normal(60, 8, n_female) # average weight 60 kg

    data = pd.DataFrame(
        {
            "Height(cm)": np.round(np.concatenate([male_height, female_height])
                , 1),
            "Weight(kg)": np.round(np.concatenate([male_weight, female_weight])
                , 1),
```

```
            "Gender": ["Male"] * n_male + ["Female"] * n_female,
        }
    )
    return data.sample(frac=1, random_state=seed).reset_index(drop=True)


# ------------------------------------------------------------
# Page setup
# ------------------------------------------------------------
st.set_page_config(page_title="K-Means Clustering", layout="wide")
st.title(" K-Means Clustering")
st.caption("Machine Learning Lab Experiment  Interactive UI (No CLI)")

# ------------------------------------------------------------
# Step 1: Read the dataset (choose it from the sidebar)
# ------------------------------------------------------------
st.sidebar.header(" Settings")

data_source = st.sidebar.radio(
    "Step 1: Choose a dataset",
    ["Height-Weight Dataset", "Upload your own CSV"],
)

has_ground_truth = False

if data_source == "Height-Weight Dataset":
    df = generate_height_weight_dataset()
    feature_cols = ["Height(cm)", "Weight(kg)"]
    label_col = "Gender" # only used later, for comparison
    has_ground_truth = True
else:
    uploaded_file = st.sidebar.file_uploader("Upload a CSV file", type=["csv
        "])
    if uploaded_file is None:
        st.info(" Upload a CSV file from the left, or choose the Height-Weight
             Dataset.")
        st.stop()
    df = pd.read_csv(uploaded_file)
    all_cols = list(df.columns)
    feature_cols = st.sidebar.multiselect(
        "Which columns are Features (must be numeric)? Pick exactly 2 for the
           cluster plot.",
        all_cols,
        default=all_cols[:2],
    )
    label_col = None

# ------------------------------------------------------------
# Step 2: K-Means parameters
# ------------------------------------------------------------
st.sidebar.markdown("---")
```

```
st.sidebar.subheader("Step 2: K-Means Parameters")
k_value = st.sidebar.slider("K (Number of clusters)", min_value=2, max_value
    =8, value=2)
random_state = st.sidebar.number_input("Random state (seed)", value=42, step
    =1)

# -------------------------------------------------------
# Dataset preview
# -------------------------------------------------------
st.subheader(" 1. Dataset Preview")
c1, c2 = st.columns([3, 1])
with c1:
    st.dataframe(df.head(10), use_container_width=True)
with c2:
    st.metric("Total Rows", df.shape[0])
    st.metric("Total Features", len(feature_cols))
    st.metric("K (clusters)", k_value)

# -------------------------------------------------------
# Step 3: Run button
# -------------------------------------------------------
st.subheader(" 2. Run K-Means")
run_clicked = st.button("Run K-Means Clustering", type="primary")

if "trained" not in st.session_state:
    st.session_state.trained = False

if run_clicked:
    if len(feature_cols) < 2:
        st.error("Please select at least 2 feature columns.")
        st.stop()

    X = df[feature_cols].to_numpy(dtype=float)

    # Feature scaling (K-Means is distance based, so scaling matters a lot)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    model = KMeans(n_clusters=k_value, random_state=int(random_state), n_init
        =10)
    cluster_labels = model.fit_predict(X_scaled)

    # Elbow method data: inertia for k = 1 .. 10
    inertias = []
    k_range = range(1, 11)
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=int(random_state), n_init=10)
        km.fit(X_scaled)
        inertias.append(km.inertia_)

    st.session_state.trained = True
```

```
    st.session_state.model = model
    st.session_state.scaler = scaler
    st.session_state.cluster_labels = cluster_labels
    st.session_state.feature_cols = feature_cols
    st.session_state.k_range = list(k_range)
    st.session_state.inertias = inertias
    st.session_state.df_result = df.copy()
    st.session_state.df_result["Cluster"] = cluster_labels
    st.session_state.has_ground_truth = has_ground_truth
    st.session_state.label_col = label_col

# ------------------------------------------------------------
# Show results
# ------------------------------------------------------------
if st.session_state.trained:
    st.success(f" K-Means finished! Found {k_value} clusters.")

    col1, col2 = st.columns(2)

    with col1:
        st.markdown("**Cluster Plot**")
        feats = st.session_state.feature_cols
        df_result = st.session_state.df_result
        fig, ax = plt.subplots()
        scatter = ax.scatter(
            df_result[feats[0]], df_result[feats[1]],
            c=st.session_state.cluster_labels, cmap="tab10", edgecolors="k",
                alpha=0.8,
        )
        centers_scaled = st.session_state.model.cluster_centers_
        centers_original = st.session_state.scaler.inverse_transform(
            centers_scaled)
        ax.scatter(
            centers_original[:, 0], centers_original[:, 1],
            c="red", marker="X", s=200, label="Centroids",
        )
        ax.set_xlabel(feats[0])
        ax.set_ylabel(feats[1])
        ax.legend()
        st.pyplot(fig)

    with col2:
        st.markdown("**Elbow Method (choosing the best K)**")
        fig2, ax2 = plt.subplots()
        ax2.plot(st.session_state.k_range, st.session_state.inertias, marker="
            o")
        ax2.set_xlabel("Number of Clusters (K)")
        ax2.set_ylabel("Inertia (within-cluster sum of squares)")
        ax2.set_title("Elbow Curve")
        st.pyplot(fig2)
        st.caption(
```

```
            "Look for the 'elbow' point  where inertia stops dropping sharply.
                "
            "That point is usually a good choice for K."
        )

    if st.session_state.has_ground_truth:
        st.markdown("**Comparing Clusters with the Real Gender Labels**")
        st.write(
            "K-Means never saw the Gender column while clustering  it only used
                 Height and Weight. "
            "This table lets us check, just for learning purposes, how well the
                 clusters line up with the real groups."
        )
        cross = pd.crosstab(
            st.session_state.df_result["Cluster"], st.session_state.df_result[
                st.session_state.label_col]
        )
        st.dataframe(cross, use_container_width=True)

    # -------------------------------------------------------
    # Step 4: Predict the cluster of new input (Live Prediction UI)
    # -------------------------------------------------------
    st.subheader(" 3. Find the Cluster for New Input")
    st.write("Type values directly into the boxes below  this is where new
        input is entered.")

    input_values = []
    cols = st.columns(len(st.session_state.feature_cols))
    for i, feat in enumerate(st.session_state.feature_cols):
        mean_v = float(df[feat].mean())
        with cols[i]:
            val = st.number_input(
                feat,
                value=round(mean_v, 1),
                step=0.1,
                format="%.1f",
            )
        input_values.append(val)

    if st.button("Find Cluster for this input"):
        X_new = st.session_state.scaler.transform([input_values])
        cluster_id = st.session_state.model.predict(X_new)[0]
        st.markdown(f"###  This point belongs to **Cluster {cluster_id}**")
else:
    st.info(" Click the 'Run K-Means Clustering' button above first.")
```

Chapter 4

Decision Tree

Introduction

A Decision Tree repeatedly asks simple yes/no questions about the features (like "20 Questions") until it can confidently name the class.

Reading the Dataset

Same Height-Weight dataset. No feature scaling is needed here, unlike KNN/SVM.

Training

```
model = DecisionTreeClassifier(
    criterion=criterion, max_depth=max_depth,
    min_samples_split=min_samples_split, random_state=42,
)
model.fit(X_train, y_train)
```

Criterion (Gini/Entropy) decides how "purity" is measured at each split. Max Depth controls how many questions deep the tree can go.

How the UI Gives Output

- Accuracy, Confusion Matrix, Classification Report
- An actual drawn tree diagram showing every question and split
- A Feature Importance chart
- Number-input boxes → Predict → Gender + probability chart

How to Run This Application

1) Keep the `dtree_app/` folder on your laptop
2) Open a terminal and go into that folder (`cd dtree_app/`)
3) Install the libraries: `pip install -r requirements.txt`

4) Run the app: `streamlit run app.py`

5) A browser tab opens automatically (if not, go to http://localhost:8501)

Implementation

```
"""
Decision Tree Classifier - Interactive UI
-----------------------------------------------------------
This is a graphical user interface built with Streamlit.
No CLI is used here  everything is done with buttons, dropdowns,
sliders, and typeable input boxes.

How to run (in your terminal):
    streamlit run dtree_app.py

Then open http://localhost:8501 in your browser.
"""

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
)

# -----------------------------------------------------------
# A simple, easy-to-understand built-in dataset:
# Predict Gender from Height and Weight
# -----------------------------------------------------------
@st.cache_data
def generate_height_weight_dataset(n=200, seed=42):
    rng = np.random.default_rng(seed)
    n_male = n // 2
    n_female = n - n_male

    male_height = rng.normal(175, 7, n_male) # average height 175 cm
    male_weight = rng.normal(78, 10, n_male) # average weight 78 kg

    female_height = rng.normal(162, 6, n_female) # average height 162 cm
    female_weight = rng.normal(60, 8, n_female) # average weight 60 kg
```

```
    data = pd.DataFrame(
        {
            "Height(cm)": np.round(np.concatenate([male_height, female_height])
                , 1),
            "Weight(kg)": np.round(np.concatenate([male_weight, female_weight])
                , 1),
            "Gender": ["Male"] * n_male + ["Female"] * n_female,
        }
    )
    return data.sample(frac=1, random_state=seed).reset_index(drop=True)


# ------------------------------------------------------------
# Page setup
# ------------------------------------------------------------
st.set_page_config(page_title="Decision Tree Classifier", layout="wide")
st.title(" Decision Tree Classifier")
st.caption("Machine Learning Lab Experiment  Interactive UI (No CLI)")

# ------------------------------------------------------------
# Step 1: Read the dataset (choose it from the sidebar)
# ------------------------------------------------------------
st.sidebar.header(" Settings")

data_source = st.sidebar.radio(
    "Step 1: Choose a dataset",
    ["Height-Weight Dataset (Gender Classification)", "Upload your own CSV"],
)

if data_source == "Height-Weight Dataset (Gender Classification)":
    df = generate_height_weight_dataset()
    feature_cols = ["Height(cm)", "Weight(kg)"]
    target_col = "Gender"
    target_names = sorted(df[target_col].unique().tolist())
else:
    uploaded_file = st.sidebar.file_uploader("Upload a CSV file", type=["csv
        "])
    if uploaded_file is None:
        st.info(" Upload a CSV file from the left, or choose the Height-Weight
             Dataset.")
        st.stop()
    df = pd.read_csv(uploaded_file)
    all_cols = list(df.columns)
    target_col = st.sidebar.selectbox("Which column is the Target/Label (what
        you want to predict)?", all_cols)
    feature_cols = st.sidebar.multiselect(
        "Which columns are Features (must be numeric)?",
        [c for c in all_cols if c != target_col],
        default=[c for c in all_cols if c != target_col],
    )
    target_names = sorted(df[target_col].astype(str).unique().tolist())
```

```
# ------------------------------------------------------------
# Step 2: Decision Tree hyperparameters
# ------------------------------------------------------------
st.sidebar.markdown("---")
st.sidebar.subheader("Step 2: Decision Tree Parameters")
criterion = st.sidebar.selectbox("Criterion (splitting rule)", ["gini", "
    entropy", "log_loss"])
max_depth = st.sidebar.slider("Max Depth (0 = unlimited)", min_value=0,
    max_value=10, value=3)
min_samples_split = st.sidebar.slider("Min samples to split a node", min_value
    =2, max_value=20, value=2)
test_size = st.sidebar.slider("Test data split (%)", min_value=10, max_value
    =50, value=20, step=5) / 100

# ------------------------------------------------------------
# Dataset preview
# ------------------------------------------------------------
st.subheader(" 1. Dataset Preview")
c1, c2 = st.columns([3, 1])
with c1:
    st.dataframe(df.head(10), use_container_width=True)
with c2:
    st.metric("Total Rows", df.shape[0])
    st.metric("Total Features", len(feature_cols))
    st.metric("Number of Classes", len(target_names))

# ------------------------------------------------------------
# Step 3: Train button
# ------------------------------------------------------------
st.subheader(" 2. Train the Model")
train_clicked = st.button("Train Decision Tree Model", type="primary")

if "trained" not in st.session_state:
    st.session_state.trained = False

if train_clicked:
    if len(feature_cols) < 1:
        st.error("Please select at least 1 feature column.")
        st.stop()

    X = df[feature_cols].to_numpy(dtype=float)
    y = df[target_col].astype(str).to_numpy()

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    depth = None if max_depth == 0 else max_depth
    model = DecisionTreeClassifier(
        criterion=criterion,
```

```
        max_depth=depth,
        min_samples_split=min_samples_split,
        random_state=42,
    )
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    st.session_state.trained = True
    st.session_state.model = model
    st.session_state.acc = acc
    st.session_state.y_test = y_test
    st.session_state.y_pred = y_pred
    st.session_state.feature_cols = feature_cols
    st.session_state.target_names = target_names

# ------------------------------------------------------------
# Show results
# ------------------------------------------------------------
if st.session_state.trained:
    st.success(f" Model trained successfully! Accuracy = {st.session_state.acc
        *100:.2f}%")

    col1, col2 = st.columns(2)

    with col1:
        st.markdown("**Confusion Matrix**")
        cm = confusion_matrix(st.session_state.y_test, st.session_state.y_pred
            )
        fig, ax = plt.subplots()
        sns.heatmap(cm, annot=True, fmt="d", cmap="Greens", ax=ax)
        ax.set_xlabel("Predicted")
        ax.set_ylabel("Actual")
        st.pyplot(fig)

    with col2:
        st.markdown("**Classification Report**")
        report = classification_report(
            st.session_state.y_test, st.session_state.y_pred, output_dict=True
        )
        st.dataframe(pd.DataFrame(report).transpose(), use_container_width=
            True)

    # ------------------------------------------------------------
    # Tree visualization
    # ------------------------------------------------------------
    st.markdown("**The Trained Decision Tree**")
    model = st.session_state.model
    fig3, ax3 = plt.subplots(figsize=(16, 8))
    plot_tree(
```

```
        model,
        feature_names=st.session_state.feature_cols,
        class_names=model.classes_,
        filled=True,
        rounded=True,
        fontsize=9,
        ax=ax3,
    )
    st.pyplot(fig3)
    st.caption(
        "Each box is a decision rule. Follow the branches down: at every step
            the tree "
        "asks a yes/no question about a feature, until it reaches a leaf with
            a final class."
    )

    # ------------------------------------------------------
    # Feature importance
    # ------------------------------------------------------
    st.markdown("**Feature Importance**")
    importance_df = pd.DataFrame(
        {"Feature": st.session_state.feature_cols, "Importance": model.
            feature_importances_}
    ).sort_values("Importance", ascending=False)
    st.bar_chart(importance_df.set_index("Feature"))
    st.caption("This shows which feature the tree relied on the most to make
        its decisions.")

    # ------------------------------------------------------
    # Step 4: Predict with new input (Live Prediction UI)
    # ------------------------------------------------------
    st.subheader(" 3. Predict with New Input")
    st.write("Type values directly into the boxes below  this is where new
        input is entered.")

    input_values = []
    cols = st.columns(len(st.session_state.feature_cols))
    for i, feat in enumerate(st.session_state.feature_cols):
        mean_v = float(df[feat].mean())
        with cols[i]:
            val = st.number_input(
                feat,
                value=round(mean_v, 1),
                step=0.1,
                format="%.1f",
            )
        input_values.append(val)

    if st.button("Predict with this input"):
        X_new = np.array([input_values])
        pred = st.session_state.model.predict(X_new)[0]
```

```
        proba = st.session_state.model.predict_proba(X_new)[0]

        st.markdown(f"###  Prediction Result: **{pred}**")
        proba_df = pd.DataFrame(
            {"Class": st.session_state.model.classes_, "Probability": proba}
        ).sort_values("Probability", ascending=False)
        st.bar_chart(proba_df.set_index("Class"))
else:
    st.info(" Click the 'Train Decision Tree Model' button above first.")
```

Chapter 5

Perceptron — Apple vs Orange (Hagan, Ch. 4)

Introduction

The Perceptron is the simplest artificial neuron. It learns through iterative trial-and-error correction, exactly as taught in Hagan's *Neural Network Design* textbook — implemented here by hand, not with a library.

Reading the Dataset

Only two prototype patterns: Orange = [Shape= 1, Texture= −1], Apple = [Shape= 1, Texture= 1], shown as an editable table.

Training — the Perceptron Learning Rule

```
n = w . p + b
a = hardlims(n) # +1 or -1
e = t - a
w_new = w_old + e * p
b_new = b_old + e
```

This repeats pattern by pattern, epoch by epoch, until a full pass produces zero errors (guaranteed for linearly separable data).

How the UI Gives Output

- A full step-by-step training log (every n, a, e, and weight update)
- Convergence message + final weights and bias
- A decision boundary plot
- Dropdowns for Shape/Texture → Classify → Apple or Orange

How to Run This Application

1) Keep the `perceptron_app/` folder on your laptop

2) Open a terminal and go into that folder (`cd perceptron_app/`)

3) Install the libraries: `pip install -r requirements.txt`

4) Run the app: `streamlit run app.py`

5) A browser tab opens automatically (if not, go to http://localhost:8501)

Implementation

```
"""
Perceptron - Apple vs Orange Recognition (Hagan, Chapter 4)
------------------------------------------------------------
This is a graphical user interface built with Streamlit.
No CLI is used here  everything is done with buttons, dropdowns,
an editable table, and typeable input boxes.

This experiment implements the classic single-neuron Perceptron
Learning Rule by hand (not sklearn), exactly the way it is taught
in Hagan's "Neural Network Design" textbook:

    n = w . p + b
    a = hardlims(n) (bipolar hard-limit: +1 or -1)
    e = t - a
    w_new = w_old + e * p
    b_new = b_old + e

How to run (in your terminal):
    streamlit run perceptron_app.py

Then open http://localhost:8501 in your browser.
"""

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

st.set_page_config(page_title="Perceptron - Apple vs Orange", layout="wide")
st.title(" Perceptron: Apple vs Orange Recognition")
st.caption("Machine Learning Lab Experiment (Hagan, Chapter 4)  Interactive UI
     (No CLI)")

st.write(
    "A simple robot has two sensors: a **Shape** sensor and a **Texture**
       sensor. "
    "Each sensor outputs a bipolar value: **+1** or **-1**. We train a single-
       neuron "
    "Perceptron to tell an Apple apart from an Orange using only these two
       sensor readings."
```

```
)

# ------------------------------------------------------------
# hardlims activation function (bipolar hard limit)
# ------------------------------------------------------------
def hardlims(n):
    return 1 if n >= 0 else -1


# ------------------------------------------------------------
# Step 1: Training patterns (editable table)
# ------------------------------------------------------------
st.subheader(" 1. Training Patterns")
st.write(
    "This is the dataset for this experiment  only two prototype patterns,
       exactly like "
    "the classic textbook example. You can edit the table below (Shape/Texture
       /Target must "
    "be +1 or -1) to match your own course slides if they use different values
       ."
)

default_data = pd.DataFrame(
    {
        "Fruit": ["Orange", "Apple"],
        "Shape": [1, 1], # +1 = round
        "Texture": [-1, 1], # -1 = rough peel, +1 = smooth
        "Target": [-1, 1], # -1 = Orange, +1 = Apple
    }
)

edited_df = st.data_editor(
    default_data,
    num_rows="dynamic",
    use_container_width=True,
    column_config={
        "Shape": st.column_config.NumberColumn("Shape (+1 Round / -1 Elongated
           )", min_value=-1, max_value=1, step=2),
        "Texture": st.column_config.NumberColumn("Texture (+1 Smooth / -1
           Rough)", min_value=-1, max_value=1, step=2),
        "Target": st.column_config.NumberColumn("Target (+1 Apple / -1 Orange)
           ", min_value=-1, max_value=1, step=2),
    },
)

# ------------------------------------------------------------
# Step 2: Learning settings
# ------------------------------------------------------------
st.sidebar.header(" Settings")
st.sidebar.subheader("Step 2: Perceptron Settings")
w1_init = st.sidebar.number_input("Initial weight w1", value=0.0, step=1.0)
```

```
w2_init = st.sidebar.number_input("Initial weight w2", value=0.0, step=1.0)
b_init = st.sidebar.number_input("Initial bias b", value=0.0, step=1.0)
max_epochs = st.sidebar.slider("Max epochs", min_value=1, max_value=20, value
    =10)

# ------------------------------------------------------------
# Step 3: Train button
# ------------------------------------------------------------
st.subheader(" 2. Train the Perceptron")
train_clicked = st.button("Train Perceptron", type="primary")

if "trained" not in st.session_state:
    st.session_state.trained = False

if train_clicked:
    patterns = edited_df[["Shape", "Texture"]].to_numpy(dtype=float)
    targets = edited_df["Target"].to_numpy(dtype=float)
    fruit_names = edited_df["Fruit"].tolist()

    w = np.array([w1_init, w2_init], dtype=float)
    b = float(b_init)

    log_rows = []
    converged = False
    converged_epoch = None

    for epoch in range(1, max_epochs + 1):
        epoch_errors = 0
        for i in range(len(patterns)):
            p = patterns[i]
            t = targets[i]
            n = float(np.dot(w, p) + b)
            a = hardlims(n)
            e = t - a
            log_rows.append(
                {
                    "Epoch": epoch,
                    "Pattern": fruit_names[i] if i < len(fruit_names) else f"P{i
                        +1}",
                    "Shape": p[0],
                    "Texture": p[1],
                    "Target": t,
                    "n = w.p+b": round(n, 2),
                    "Output a": a,
                    "Error e": e,
                    "w1": round(w[0] + e * p[0], 2),
                    "w2": round(w[1] + e * p[1], 2),
                    "b": round(b + e, 2),
                }
            )
            w = w + e * p
```

```
            b = b + e
            if e != 0:
                epoch_errors += 1
        if epoch_errors == 0:
            converged = True
            converged_epoch = epoch
            break

    st.session_state.trained = True
    st.session_state.log_df = pd.DataFrame(log_rows)
    st.session_state.final_w = w
    st.session_state.final_b = b
    st.session_state.converged = converged
    st.session_state.converged_epoch = converged_epoch
    st.session_state.patterns = patterns
    st.session_state.targets = targets
    st.session_state.fruit_names = fruit_names

# ------------------------------------------------------------
# Show results
# ------------------------------------------------------------
if st.session_state.trained:
    if st.session_state.converged:
        st.success(
            f" The Perceptron converged after {st.session_state.converged_epoch
                } epoch(s)! "
            f"Final weights: w1 = {st.session_state.final_w[0]:.2f}, "
            f"w2 = {st.session_state.final_w[1]:.2f}, b = {st.session_state.
                final_b:.2f}"
        )
    else:
        st.warning(
            " The Perceptron did not fully converge within the max epoch limit.
                 "
            "Try increasing 'Max epochs', or check that the patterns are
                linearly separable."
        )

    st.markdown("**Step-by-Step Training Log (Weight Updates)**")
    st.dataframe(st.session_state.log_df, use_container_width=True, hide_index
        =True)
    st.caption(
        "Each row is one training pattern seen by the Perceptron. Whenever
            Error e is not 0, "
        "the weights and bias are updated using: w_new = w_old + ep, b_new =
            b_old + e."
    )

    # ------------------------------------------------------
    # Decision boundary plot
    # ------------------------------------------------------
```

```
    st.markdown("**Decision Boundary**")
    w = st.session_state.final_w
    b = st.session_state.final_b
    patterns = st.session_state.patterns
    targets = st.session_state.targets

    fig, ax = plt.subplots()
    colors = ["orange" if t == -1 else "red" for t in targets]
    ax.scatter(patterns[:, 0], patterns[:, 1], c=colors, s=200, edgecolors="k
        ", zorder=3)
    for i, name in enumerate(st.session_state.fruit_names):
        ax.annotate(name, (patterns[i, 0], patterns[i, 1]), textcoords="offset
            points", xytext=(10, 10))

    x_vals = np.linspace(-2, 2, 100)
    if w[1] != 0:
        y_vals = -(w[0] * x_vals + b) / w[1]
        ax.plot(x_vals, y_vals, "b--", label="Decision Boundary")
    elif w[0] != 0:
        ax.axvline(-b / w[0], color="b", linestyle="--", label="Decision
            Boundary")

    ax.set_xlim(-2, 2)
    ax.set_ylim(-2, 2)
    ax.axhline(0, color="gray", linewidth=0.5)
    ax.axvline(0, color="gray", linewidth=0.5)
    ax.set_xlabel("Shape (-1 = Elongated, +1 = Round)")
    ax.set_ylabel("Texture (-1 = Rough, +1 = Smooth)")
    ax.legend()
    st.pyplot(fig)

    # ------------------------------------------------------
    # Step 4: Live classification with new input
    # ------------------------------------------------------
    st.subheader(" 3. Classify New Input")
    st.write("Choose sensor readings below  this is where new input is entered
        .")

    col1, col2 = st.columns(2)
    with col1:
        shape_choice = st.selectbox("Shape sensor", ["+1 (Round)", "-1 (
            Elongated)"])
    with col2:
        texture_choice = st.selectbox("Texture sensor", ["+1 (Smooth)", "-1 (
            Rough)"])

    shape_val = 1 if shape_choice.startswith("+1") else -1
    texture_val = 1 if texture_choice.startswith("+1") else -1

    if st.button("Classify this input"):
        n = float(np.dot(w, [shape_val, texture_val]) + b)
```

```
        a = hardlims(n)
        result = " Apple" if a == 1 else " Orange"
        st.markdown(f"###  Prediction Result: **{result}**")
        st.caption(f"n = w.p + b = {n:.2f}  a = hardlims(n) = {a}")
else:
    st.info(" Click the 'Train Perceptron' button above first.")
```